# TriMem: A Cardinality-Aware Tripartite Memory Architecture with Deterministic Conflict Resolution for Conversational AI Agents

Amine Essahraoui, Mohammed Amine Zbida, Mohammed Amine Hssaine
*Students, Artificial Intelligence Engineering Program*
École Nationale Supérieure d'Arts et Métiers (ENSAM), Meknès, Morocco
amine.essahraoui@email.com

***Abstract*—Persistent, personalized memory remains an open engineering problem for conversational AI agents deployed across multiple sessions. Naive Retrieval-Augmented Generation (RAG) treats all prior interaction history as an undifferentiated pool of text chunks, which produces three recurring failure modes: (i) stale facts compete on equal footing with their up-to-date replacements, (ii) storage grows unbounded as raw transcripts accumulate, and (iii) behavioral instructions have no privileged status over incidental conversational content. We present TriMem, an open-source, cognitively-inspired memory architecture that partitions agent context into three interacting subsystems: *Episodic Memory* (dual vector/keyword storage of raw interactions with exponential recency-importance decay), *Semantic Memory* (a cardinality-typed knowledge graph that overwrites one-to-one facts and appends one-to-many facts), and *Procedural Memory* (a three-tier hierarchy of behavioral Standard Operating Procedures). These subsystems are unified by an explicit, deterministic *Ground Truth Hierarchy* that resolves cross-memory conflicts at context-assembly time, together with an LLM-driven consolidation pipeline that periodically compresses episodic traces into semantic facts. We describe the architecture, its cardinality-aware storage model, and its conflict-resolution and consolidation algorithms in detail, and report results from an internal validation suite covering identity persistence, semantic overwrite correctness, procedural enforcement, multi-user isolation, and episodic compression (observed ratio $\rho \approx 0.60$). We position TriMem relative to recent production and research memory systems (MemGPT/Letta, Mem0, Zep, A-Mem, MemoryBank) and identify the aspects of our internal evaluation that fall short of benchmark-grade evidence (e.g., LongMemEval, LoCoMo), which we outline as concrete next steps. We argue that TriMem's primary contribution is architectural and systems-oriented rather than a new learning algorithm: a reproducible, openly documented pattern for combining typed memory stores with deterministic precedence rules.**

***Index Terms*—Agentic AI, long-term memory, knowledge graphs, retrieval-augmented generation, memory consolidation, conflict resolution**

## I. Introduction

The fixed context window of Large Language Models (LLMs) fundamentally restricts the development of personalized, longitudinal conversational agents. The dominant workaround, flat Retrieval-Augmented Generation (RAG) [3], treats historical interactions as an unstructured, undifferentiated collection of text segments retrieved by embedding similarity. This design choice is adequate for single-shot document question answering, but it degrades in longitudinal, multi-session agent settings for three concrete reasons.

First, *stale-fact competition*: when a user updates a piece of information (e.g., changing a preference, a code, or a role), the old and new statements are both present in the vector store and compete for retrieval on essentially equal footing, since both are semantically similar to a query about that fact. Second, *unbounded accumulation*: every turn of dialogue is stored verbatim, so storage and retrieval latency grow monotonically with conversation length, even though the overwhelming majority of turns (greetings, clarifying questions, small talk) carry no durable information. Third, *undifferentiated priority*: an explicit behavioral instruction from the user ("always answer in French") has no distinguished status relative to an incidental remark, so nothing in the architecture guarantees that instructions are honored consistently.

Human cognition does not address the analogous problem by accumulating flat transcripts. Cognitive and neuroscience models of long-term memory instead distinguish *episodic* memory (autobiographical experience, indexed by time and context), *semantic* memory (context-free factual knowledge), and *procedural* memory (skills and behavioral routines) [11]. We take this tripartite decomposition as a design template rather than a literal biological claim, and use it to motivate three separate storage and retrieval mechanisms, each matched to the access pattern of the information it holds.

We present **TriMem**, an implemented, open architecture that operationalizes this decomposition for LLM-based conversational agents. Rather than proposing a new learning algorithm, TriMem's contribution is at the level of *systems architecture*: an explicit, deterministic mechanism for resolving conflicts between memory types, a cardinality-typed ontology that structurally prevents duplicate or contradictory semantic facts, and an LLM-driven consolidation loop that keeps episodic storage bounded. We built and instrumented a working implementation (FastAPI, ChromaDB, SQLite FTS5, Groq-hosted Llama-3.1-8B, Ollama-hosted `nomic-embed-text`) and report results from an internal validation suite designed to stress-test each of these mechanisms.

We are explicit about the scope of the evidence we provide. Our validation suite exercises specific, targeted scenarios (fact

overwrite, multi-user isolation, procedural enforcement, consolidation) on a small number of synthetic users, and is not a substitute for evaluation on standardized long-term-memory benchmarks such as LongMemEval [10] or LoCoMo [9] against strong production baselines such as Mem0 [5] or Zep [6]. Section XIII details this gap and the benchmark-based evaluation we consider a prerequisite for any claim of state-of-the-art retrieval accuracy.

Our contributions are:

1) **A cardinality-aware semantic ontology.** We classify factual relations as one-to-one (overwrite semantics) or one-to-many (append-with-deduplication semantics), which structurally prevents the "cumulative fact" problem that affects undifferentiated triplet stores and vector indexes (Section VI).
2) **An explicit Ground Truth Hierarchy.** A deterministic, architecturally-enforced precedence rule (Procedural $>$ Semantic $>$ Episodic) applied at context-assembly time, rather than left to the LLM's implicit judgment (Section VIII).
3) **A dual-storage hybrid retrieval mechanism for episodic memory** that combines HNSW-indexed vector similarity with SQLite FTS5 keyword search, mitigating the respective blind spots of each retrieval mode (Section V).
4) **An LLM-driven consolidation pipeline** that periodically distills episodic traces into semantic facts and prunes redundant storage, with an observed compression ratio of $\rho \approx 0.60$ on our test conversations (Section IX).
5) **An open, reproducible reference implementation** together with an internal validation suite and an honest account of its limitations relative to benchmark-grade evaluation (Sections XI–XIII).

The remainder of this paper is organized as follows. Section II situates TriMem relative to recent agent-memory systems. Section III formalizes the conflict-resolution problem we address. Section IV describes the overall system architecture. Sections V–VII detail each memory subsystem. Section VIII formalizes the Ground Truth Hierarchy and context-assembly procedure. Section IX describes memory consolidation. Section XI reports our internal validation results, Section XII discusses them, Section XIII enumerates limitations and threats to validity, and Section XV concludes.

## II. Related Work

**Reflection-based episodic agents.** *Generative Agents* [1] introduced reflection mechanisms that synthesize higher-level inferences from a flat stream of episodic observations, weighted by an LLM-assigned importance score and a recency decay term. TriMem's episodic scoring function (Eq. 1) is directly inspired by this design, but Generative Agents does not distinguish episodic experience from a separate, overwritable factual store, so updated facts remain one vector among many rather than a privileged, singular entry.

**Operating-system-inspired context management.** MemGPT (now Letta) [2] models the LLM's context window as a virtual-memory hierarchy, paging content between a bounded "main context" and an unbounded "archival" store under interrupt-driven control. This provides an elegant mechanism for *what fits in context*, but, as noted in subsequent comparative work [12], it offers no principled mechanism for confidence-weighted forgetting or for resolving contradictions between archival entries.

**Production memory layers.** Mem0 [5] is closely related in spirit: it extracts salient facts from conversation turns and uses an LLM-based routing controller to decide, per candidate fact, whether to `ADD`, `UPDATE`, `DELETE`, or discard it relative to the top-$k$ most similar existing memories. This is a more general, learned analogue of TriMem's cardinality-typed overwrite rule, at the cost of requiring a routing LLM call per candidate fact and leaving the update decision to model judgment rather than a deterministic schema. Mem0$^g$ extends this with an optional Neo4j graph layer. Zep [6] similarly layers a temporal knowledge graph on top of dense retrieval, explicitly optimizing for multi-session temporal reasoning; unlike TriMem, temporal graph edges in Zep are validity-timestamped rather than partitioned into a hard one-to-one/one-to-N cardinality typing.

**Dynamic and self-organizing memory.** A-Mem [7] takes a Zettelkasten-inspired approach in which memory notes are dynamically linked and can be retroactively refined ("memory evolution"), and MemoryBank [8] introduces update and forgetting mechanisms explicitly inspired by the Ebbinghaus forgetting curve for long-term personalized dialogue. Both systems address staleness and unbounded growth, but, like Mem0, delegate the update/forgetting decision to a single learned or heuristic mechanism rather than separating it into distinct memory types with independent cardinality and priority semantics.

**Benchmarks.** LoCoMo [9] and LongMemEval [10] provide standardized multi-session dialogue benchmarks with single-hop, multi-hop, and temporal-reasoning question categories, and are the evaluation protocols against which Mem0 and related systems report headline numbers. We did not evaluate TriMem on these benchmarks; we return to this gap in Section XIII.

**Positioning.** Table I summarizes the comparison along four structural dimensions. TriMem's distinguishing design choice is not any single mechanism in isolation — dual-storage episodic retrieval, cardinality-typed facts, and importance-gated extraction each have precedent — but their combination under one *explicit, deterministic* precedence rule that is enforced at prompt-assembly time rather than learned or left implicit, together with a fully open, inspectable reference implementation.

## III. Problem Formulation

We formalize the cross-memory conflict that motivates the Ground Truth Hierarchy. Let a user $u$ be associated with an episodic store $E_u = \{e_1, \ldots, e_k\}$ (raw interaction traces) and a semantic store $F_u = \{f_1, \ldots, f_n\}$ (extracted facts, each a triplet $(\text{subject}, \text{relation}, \text{object})$). For a query $q$ issued

TABLE I: Structural comparison with related agent-memory systems

| System | Typed cardinality | Explicit priority rule | Dual-index episodic | Benchmarked |
|---|---|---|---|---|
| MemGPT/Letta [2] | ✗ | ✗ | ✗ | ✗ |
| Mem0 [5] | Learned | ✗ | ✗ | ✓ (LoCoMo) |
| Zep [6] | Temporal | ✗ | ✗ | ✓ |
| A-Mem [7] | ✗ | ✗ | ✗ | ✓ |
| MemoryBank [8] | ✗ | ✗ | ✗ | partial |
| **TriMem (this work)** | **Explicit (1:1 / 1:N)** | **Deterministic** | ✓ | **Not yet (internal only)** |

at time $t_{\text{now}}$, a naive RAG system retrieves a ranked union $R(q) \subseteq E_u \cup F_u$ ordered purely by embedding similarity to $q$. A *conflict* occurs when two retrieved items $r_i, r_j \in R(q)$ assert incompatible values for the same relation (e.g., $e_i$ mentions an old vault code, while $f_j$ records the current one). Under flat RAG, resolving this conflict is delegated entirely to the LLM's implicit judgment at generation time, with no architectural guarantee that the more recent or more authoritative source wins.

TriMem instead defines a strict typing and precedence function. Each relation $r$ is assigned a cardinality type $\kappa(r) \in \{1{:}1, 1{:}N\}$ at the ontology level (Section VI), and each memory subsystem $M \in \{\text{Procedural}, \text{Semantic}, \text{Episodic}\}$ is assigned a fixed precedence $\pi(M)$ with $\pi(\text{Procedural}) > \pi(\text{Semantic}) > \pi(\text{Episodic})$. Context assembly (Section VIII) enforces that, whenever two memories reference the same underlying fact, the item from the subsystem with higher $\pi$ is presented to the LLM as authoritative, and for 1:1 relations within Semantic Memory itself, insertion of a new fact structurally removes the previous one rather than merely outranking it. This converts conflict resolution from a soft, learned preference into a hard architectural invariant for the 1:1 case, and a strong prompt-level precedence for the cross-subsystem case.

## IV. System Architecture

TriMem follows a layered service architecture with four principal tiers: Presentation/API, Core Services, Memory Subsystems, and Infrastructure (Table II, Figure 1). Each memory subsystem is implemented as a singleton-scoped Python class, so a single process holds consistent state while user data remains logically partitioned by `user_id` at the storage layer.

Table III lists the concrete technology mapped to each responsibility. The choice of a locally-hosted embedding model (`nomic-embed-text` via Ollama) for the highest-frequency operation (vectorization) is a deliberate cost/latency trade-off: it removes network round-trips and per-call cost from the read path, while the comparatively rarer, heavier operations (generation, classification, extraction) are offloaded to Groq's LPU-based inference infrastructure for low-latency generation.

Figure 2 shows the end-to-end request lifecycle. Each user message triggers a synchronous *read path* (embedding, hybrid retrieval across all three subsystems, context assembly, generation) followed by an asynchronous *write path* (importance scoring, episodic persistence, conditional semantic/procedural extraction, consolidation check) executed as a FastAPI background task, so that memory-write latency never delays the conversational response.

TABLE II: TriMem system architecture layers

| Layer | Components | Responsibility |
|---|---|---|
| Presentation / API | FastAPI router, Pydantic models | HTTP handling, validation, response formatting |
| Core Services | LLM client, embedding engine, config | Text generation, vectorization, environment management |
| Memory Subsystems | Episodic, Semantic, Procedural, Consolidation | Storage, retrieval, and compression of multi-modal memory |
| Infrastructure | ChromaDB, SQLite, Groq, Ollama | Persistent storage and external model inference |

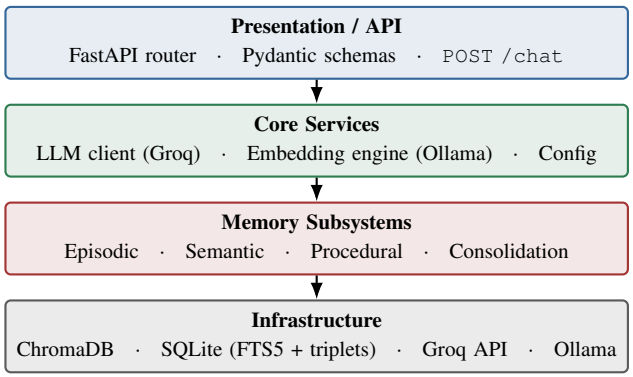



Fig. 1: Layered system architecture. Requests flow top-down; each layer depends only on the layer below it.

## V. Episodic Memory

### A. Dual-Storage Design

The `EpisodicMemory` module writes every interaction to two backends simultaneously: (i) ChromaDB, storing the interaction text, its embedding vector, and metadata (`user_id`, timestamp, importance score), indexed with HNSW under cosine distance for approximate nearest-neighbor search; and (ii) a SQLite FTS5 virtual table storing the same text for

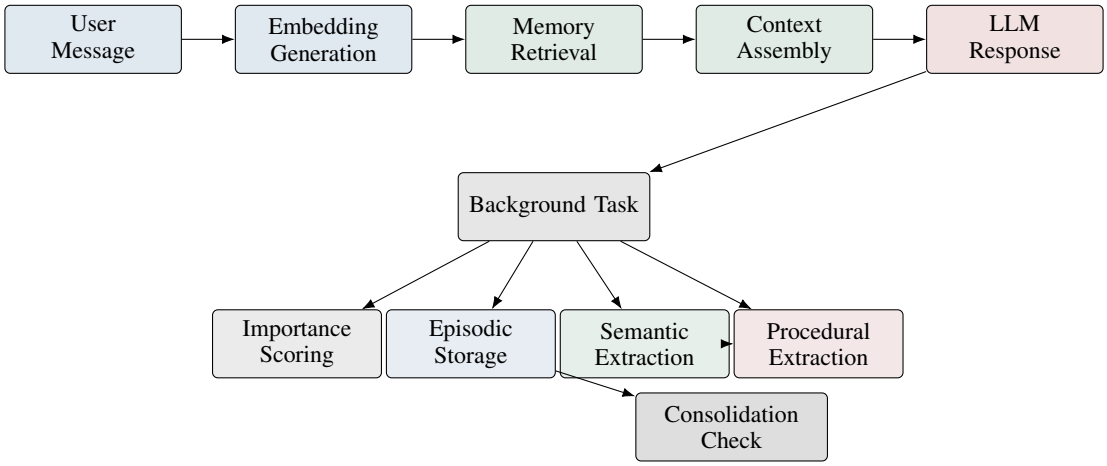



Fig. 2: End-to-end data flow. The read path (top row) is synchronous; the write path (bottom) runs as a background task after the response has been returned to the user.

TABLE III: Technology stack and component mapping

| Component | Technology | Purpose |
|---|---|---|
| Web framework | FastAPI | REST endpoint for chat |
| Data validation | Pydantic v2 | Request/response schema validation |
| Vector database | ChromaDB (persistent) | Semantic search: episodic & procedural |
| Full-text search | SQLite FTS5 | Keyword search for episodic memory |
| Knowledge graph | SQLite (triplets) | Cardinality-typed semantic fact storage |
| Embedding model | Ollama `nomic-embed-text` | 768-dim text vectorization |
| LLM inference | Groq, Llama-3.1-8B-instant | Generation, scoring, extraction |
| Visualization | PyVis | Interactive knowledge-graph rendering |

exact keyword and boolean-OR matching. This dual-storage invariant means that, for every interaction $i$, $i \in C \cap S$ (where $C$ is the ChromaDB collection and $S$ the FTS5 table) until consolidation removes $i$ from $C$.

### B. Hybrid Retrieval

Algorithm 1 merges results from both backends, deduplicating by content so that an interaction discovered by both vector and keyword search is scored once. Vector (semantic) matches are inserted first, so keyword search only contributes items not already found by embedding similarity — a deliberate choice that lets keyword search recover rare-term or exact-string matches (e.g., a numeric code) that embedding similarity can under-rank.

**Algorithm 1** Hybrid Episodic Retrieval

**Require:** user_id, query_embedding, query_text, $n$
**Ensure:** Ranked context string
$U \leftarrow \{\}$ {unique memories, keyed by content}
$R_{\text{vec}} \leftarrow$ ChromaDB.query(query_embedding, user_id, $n$)
**for** $(doc, meta) \in R_{\text{vec}}$ **do**
  $U[doc] \leftarrow S(doc)$ {Eq. 1}
**end for**
$R_{\text{kw}} \leftarrow$ FTS5.match(sanitize(query_text), user_id)
**for** $(t, I, content) \in R_{\text{kw}}$ **do**
  **if** $content \notin U$ **then**
    $U[content] \leftarrow S(content)$
  **end if**
**end for**
**return** top-$n$ of $U$ sorted by score, descending

### C. Temporal Decay

Retrieved episodic memories are ranked by a score combining LLM-assigned importance with exponential recency decay:

$$S(m) = I_m \cdot \exp\left(-\lambda \cdot \frac{t_{\text{now}} - t_m}{86400}\right) \tag{1}$$

where $I_m \in [1, 10]$ is the importance score, $t_m$ the memory's timestamp, and $\lambda = 0.02$ the daily decay constant, chosen empirically to give a half-life of approximately 35 days for a minimally-important ($I_m = 1$) memory while leaving high-importance memories ($I_m$ near 10) relevant for several months. This is the same decay-with-importance principle used by *Generative Agents* [1], adapted to a dual-index retrieval setting.

## VI. Semantic Memory

### A. Cardinality-Typed Ontology

Semantic Memory stores subject-relation-object triplets in SQLite, with subject fixed to `User` and relation drawn from a closed ontology of 17 relations across six categories (Table IV). Constraining the LLM extractor to a closed vocabulary

TABLE IV: Semantic memory relation ontology

| Category | Relations | Card. |
|---|---|---|
| Identity & demographics | HAS_NAME, HAS_NICKNAME, HAS_CODE, LIVES_IN | 1:1 |
| | SPEAKS_LANGUAGE | 1:$N$ |
| Academic & professional | STUDIES_AT, HAS_ROLE | 1:1 |
| | WORKS_AT | 1:$N$ |
| Technical & capabilities | USES_OS | 1:1 |
| | IS_A, USES_TOOL, HAS_SKILL, WANTS_TO_LEARN | 1:$N$ |
| Projects & ambitions | WORKS_ON, HAS_GOAL | 1:$N$ |
| Personal preferences | INTERESTED_IN, HAS_HOBBIES | 1:$N$ |

TABLE V: SOP scope classification

| Scope | Activation | Example |
|---|---|---|
| global | Every interaction, unconditionally | "Always respond in French" |
| task_specific | Query semantically matches task tag | "When writing code, use OOP" |
| universal | Fallback, no personal rule applies | System-defined base guidelines |

**Algorithm 2** Hierarchical SOP Retrieval

**Require:** query embedding $q$, user_id $u$
1: $G \leftarrow$ fetch all global SOPs for $u$ (metadata filter)
2: $T \leftarrow$ vector-search task_specific SOPs for $u$, dist $< 0.60$
3: $V \leftarrow$ vector-search universal SOPs, dist $< 0.60$
4: **return** concatenate$(G, T, V)$ with priority section headers

at extraction time prevents hallucinated relation types and keeps the graph schema stable.

### B. Overwrite and Append Semantics

For 1:1 relations (e.g., HAS_NAME, HAS_CODE), a new fact triggers a delete-then-insert transaction, guaranteeing at most one active value per user per relation:

```
DELETE FROM triplets WHERE user_id=? AND
relation=?;
INSERT INTO triplets (...) VALUES (...);
```

For 1:$N$ relations (e.g., HAS_SKILL), a unique constraint on (user_id, relation, object) allows accumulation while silently rejecting exact duplicates. This mechanism directly targets the "cumulative fact" problem: without it, an updated name or code would coexist with its predecessor as two independently-retrievable vectors, and the LLM would have to arbitrate between them at generation time with no guarantee of choosing the correct one.

### C. Vector-Enhanced Retrieval

Rather than exact keyword lookup, semantic facts are converted to natural-language propositions (e.g., (HAS_NAME, "Alexander") $\rightarrow$ "The user's real name is Alexander") and embedded with the asymmetric search_document: prefix; queries are embedded with the search_query: prefix, and retrieval ranks facts by cosine similarity:

$$\text{sim}(q, f) = \frac{\sum_{k=1}^{n} q_k f_k}{\|q\| \|f\|} \tag{2}$$

Facts below a similarity threshold (0.50–0.60, tuned per deployment) are discarded. This lets the system match, for instance, "What OS do I use?" to the stored proposition "The user uses the operating system Linux," which a pure keyword match would miss.

## VII. Procedural Memory

Procedural memory encodes Standard Operating Procedures (SOPs) — persistent behavioral instructions — classified into three scopes (Table V) and retrieved through a fixed three-tier cascade (Algorithm 2): all personal global SOPs are fetched unconditionally via metadata filter; personal task_specific SOPs are retrieved by vector search filtered to the current query's task category; and administrator-defined universal SOPs, loaded once at startup under a reserved user_id, act as a behavioral fallback. A cosine-distance threshold of 0.60 is applied to both vector-searched tiers to keep marginally relevant rules out of context.

For task_specific SOPs, the searchable document is enriched with an explicit task-category prefix ("Task category: {tag}. Action to apply: {instruction}") before embedding, which we found improves retrieval precision by making the task category an explicit lexical signal in the embedding space rather than something the embedding model must infer implicitly.

## VIII. The Ground Truth Hierarchy and Context Assembly

Context assembly is the point at which the three subsystems' outputs are combined into a single prompt and where conflicts between them must be resolved. TriMem enforces a strict precedence, illustrated in Figure 3: Procedural rules are checked first and applied unconditionally to response format; Semantic facts are labeled as absolute, most-up-to-date ground truth; Episodic context is presented as background only, to be used when it adds genuine value and discarded otherwise.

Concretely, the assembled prompt is partitioned into three explicitly labeled sections:

```
=== FACTS (Semantic) === {semantic_context}
=== BEHAVIORAL RULES (Procedural) ===
{procedural_context}
```

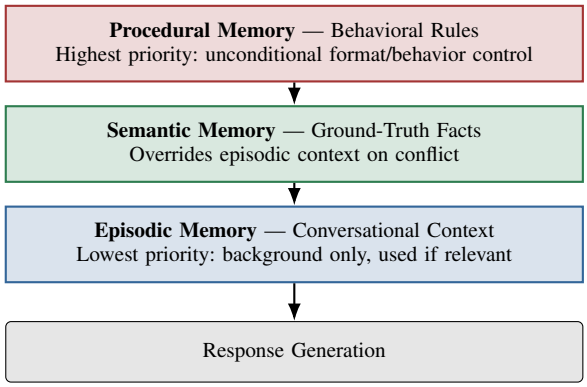



Fig. 3: Ground Truth Hierarchy: precedence enforced at context-assembly time.

```
=== PAST CONTEXT (Episodic) ===
{episodic_context}
```

together with seven explicit instruction rules governing their use: (1) ground-truth override — semantic facts take precedence over all other context; (2) procedural-first — behavioral rules are applied before response formulation; (3) relevance filtering — episodic context is used only if it adds value; (4) no invention — the model must not fabricate facts or memories; (5) user isolation — no cross-user data may be referenced; (6) a fixed tone specification; and (7) no exposed chain-of-thought in the output.

We emphasize that, for the cross-subsystem case, this precedence is a strong prompt-level instruction rather than a hard architectural guarantee — it relies on instruction-following by the underlying LLM, unlike the 1:1 overwrite mechanism in Semantic Memory (Section VI), which is structurally enforced at the storage layer regardless of model behavior. This distinction matters for interpreting our evaluation results (Section XI) and is discussed further in Section XIII.

## IX. Memory Consolidation

### A. Trigger and Pipeline

To keep episodic storage bounded, a per-user interaction counter triggers consolidation once it reaches a configurable threshold $N$ (Algorithm 3, Figure 4). Consolidation fetches the $N$ most recent episodic interactions, sends them to the LLM alongside the current semantic fact set, and asks it to classify each interaction as *redundant* (all meaningful content already captured in Semantic Memory) or *valuable* (containing nuance or context not yet extracted). Redundant interactions are deleted from ChromaDB; valuable ones trigger re-extraction into Semantic Memory.

### B. Compression Ratio

We define the compression ratio over an evaluated batch $E$ as

$$\rho = \frac{|\{m \in E \mid \text{redundant}(m)\}|}{|E|} \tag{3}$$

On our internal test conversations (Section XI), $\rho \approx 0.60$: three of five seeded interactions were classified redundant and

**Algorithm 3** Consolidation Pass

**Require:** user_id $u$, threshold $N$

count$[u] \leftarrow$ count$[u] + 1$
**if** count$[u] \geq N$ **then**
  $E' \leftarrow N$ most recent episodic traces for $u$
  $F \leftarrow$ current semantic facts for $u$
  **for** $m \in E'$ **do**
    **if** LLM.is_redundant$(m, F)$ **then**
      delete $m$ from ChromaDB
    **else**
      re-extract semantic facts from $m$
    **end if**
  **end for**
  count$[u] \leftarrow 0$
**end if**

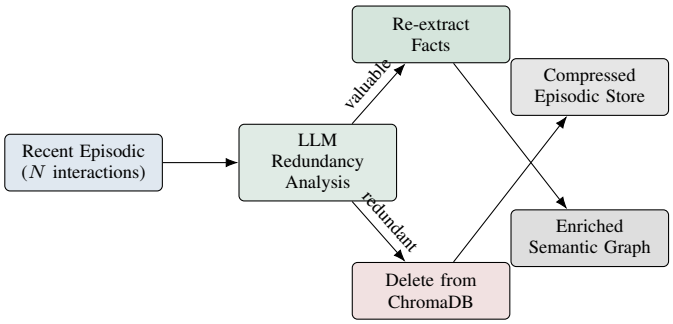



Fig. 4: Memory consolidation pipeline.

removed, while the remaining 40% — judged to carry contextual nuance beyond what had been extracted into Semantic Memory — were retained. We report this as a single internal measurement, not a statistically characterized distribution; Section XIII discusses what a rigorous treatment of this metric would require.

## X. Implementation Notes

The reference implementation is organized as a Python package under `app/` with sub-packages `api`, `core`, and `memory`. The importance-scoring function `get_importance_score` implements a four-tier rubric (Table VI) that gates downstream extraction: only interactions scored $\geq 6$ trigger semantic and procedural extraction, while all interactions (regardless of score) are stored episodically. In our test data, this meant roughly 40% of interactions triggered full extraction, substantially reducing noise injected into the semantic and procedural stores.

Two design patterns are notable at the API layer: (1) a *background-task pattern*, in which memory writes are deferred via FastAPI's `BackgroundTasks` so the HTTP response returns immediately after generation; and (2) *context-conditional prompting*, in which the system prompt differs depending on whether prior context exists for the user (a first-interaction welcome prompt versus the full three-section context template of Section VIII).

TABLE VI: Importance scoring rubric

| Score | Category | Extraction trigger |
|---|---|---|
| 1–3 | Noise & inquiry | Episodic only |
| 4–5 | Ephemeral | Episodic only |
| 6–8 | Profiling | Episodic + Semantic + Procedural |
| 9–10 | Critical | Episodic + Semantic + Procedural |

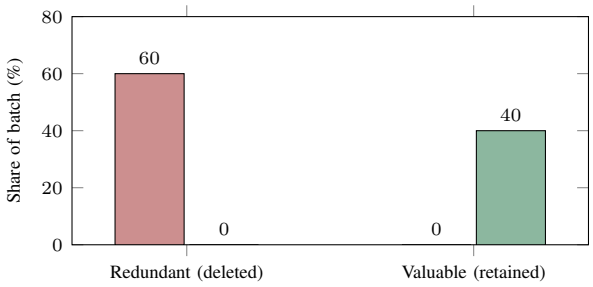



Fig. 5: Observed consolidation split on the seeded five-interaction test batch ($\rho \approx 0.60$). Single-batch measurement; not a benchmark result.

## XI. Internal Evaluation

### A. Experimental Setup

We validated TriMem with a purpose-built internal test suite (Table VII) exercising three synthetic user identities across five test files, targeting each subsystem individually and the full pipeline in integration. This is deliberately described as an *internal validation suite* rather than a benchmark: it verifies that each mechanism behaves as designed on scenarios we constructed to exercise it, and does not measure retrieval accuracy against an independent, standardized question set.

### B. Results

**Semantic overwrite.** Across repeated updates to 1:1 relations (vault code, operating system, role), the delete-then-insert mechanism correctly replaced the prior value without creating duplicate rows, confirmed by direct database inspection after each update.

**Semantic retrieval quality.** Cosine similarity scores for facts judged relevant by the test author ranged from $0.51$ to $0.76$; scores below the $0.6$ default threshold were excluded from context, at the cost of occasionally omitting borderline-relevant facts (a precision/recall trade-off we did not tune systematically).

**Procedural enforcement.** Both `global` rules (e.g., a language preference) and `task_specific` rules (e.g., a coding-style instruction) were correctly surfaced and, in manual inspection of generated responses, honored by the LLM in the test scenarios exercised.

**User isolation.** Queries about one synthetic user's data returned no information when issued under a different `user_id`, consistent with `user_id` filtering being applied at every query path.

**Consolidation.** As reported in Section IX, $\rho \approx 0.60$ on the seeded five-interaction batch, with the retained 40% judged (by manual inspection) to carry information the extracted semantic facts did not.

**Known issues.** Two HTTP 500 errors were observed during role-update tests for one synthetic user, indicating an edge case in the semantic extraction pipeline when processing promotion-style messages (e.g., a role changing while other attributes are mentioned in the same turn). Separately, in a run with consolidation disabled, an updated vault code briefly coexisted with its predecessor in episodic retrieval output, confirming that the Ground Truth Hierarchy's prompt-level precedence — rather than storage-level deletion — is what prevents the stale episodic value from being treated as authoritative, and that this precedence depends on instruction-following by the underlying LLM. We report both issues here rather than omitting them, since they directly bound the strength of the claims in Section VIII.

Figure 5 visualizes the observed consolidation outcome on the seeded batch as a simple two-category split; it is a single measurement on one small batch and should be read as illustrative, not as a statistically powered result.

## XII. Discussion

The evidence gathered supports a narrower and more defensible claim than "TriMem eliminates hallucinations": within the scenarios we constructed, the cardinality-typed overwrite mechanism reliably prevents duplicate 1:1 facts at the storage layer, and the Ground Truth Hierarchy's prompt-level precedence was sufficient, in the cases we tested, to make the LLM prefer the semantic fact over a conflicting episodic trace. The former is a structural guarantee; the latter is an empirical, model-dependent observation that could degrade with a different underlying LLM, a longer context, or an adversarially phrased query, none of which we tested.

We view TriMem's main contribution as architectural rather than algorithmic: no single mechanism here is individually novel (importance-weighted decay follows *Generative Agents* [1]; LLM-driven update/redundancy classification is close in spirit to Mem0's routing controller [5]), but the combination — a hard cardinality typing for facts, a soft but explicit precedence rule across memory types, and a dual-index episodic store — is packaged as a fully open, documented reference implementation, which we believe is useful independent of whether it advances the state of the art on standardized retrieval benchmarks.

## XIII. Limitations and Threats to Validity

We enumerate limitations explicitly, several of which we consider prerequisites for any stronger claim of effectiveness.

1) **No standardized benchmark evaluation.** We did not evaluate TriMem on LongMemEval [10] or LoCoMo [9], nor did we compare retrieval accuracy, la-

TABLE VII: Internal test suite coverage

| Test file | Focus area | Key validations |
|---|---|---|
| `test_globale_finale.py` | Full integration, 3 users | Identity, retrieval, procedural enforcement, overwrite, isolation |
| `test_scorer.py` | Importance scoring | 28 messages across 4 tiers; latency profiling |
| `test_semantic.py` | Semantic extraction | 12 sequences; 1:1 overwrite; 1:$N$ append; restraint test |
| `test_procedural.py` | Procedural extraction/retrieval | Multi-user isolation; conflicting rules; hierarchical retrieval |
| `test_consolidation.py` | Consolidation | Redundancy detection; compression ratio measurement |

tency, or token cost against Mem0 [5], Zep [6], or a flat-RAG baseline under matched conditions. The 100%-style success claims one might be tempted to draw from our internal tests should not be read as benchmark-grade accuracy figures; they describe behavior on scenarios we designed to exercise specific mechanisms, at small scale (3 synthetic users, single-digit interaction counts per test file).

2) **No ablation study.** We have not isolated the individual contribution of dual-storage retrieval, cardinality typing, or the Ground Truth Hierarchy prompt versus one another; it is possible that a subset of these mechanisms accounts for most of the observed behavior.
3) **Prompt-level, not architectural, cross-subsystem precedence.** As noted in Section VIII, the Procedural $>$ Semantic $>$ Episodic ordering across subsystems is enforced by instructing the LLM, not by removing conflicting episodic content from the prompt. Only the within-Semantic-Memory 1:1 overwrite is a hard storage-level guarantee. This distinction was not clearly separated in our earlier reporting and is corrected here.
4) **Single point of failure.** Groq is a hard dependency for generation, importance scoring, fact extraction, SOP extraction, and consolidation; an outage disables all memory intelligence beyond raw episodic storage. No fallback LLM backend is implemented.
5) **No pruning in Semantic Memory.** 1:$N$ relations (skills, hobbies, interests) accumulate indefinitely with no forgetting or summarization mechanism, which will eventually degrade retrieval precision for long-lived users.
6) **Limited consolidation scope.** Consolidation only examines the most recent $N$ interactions rather than the full episodic history, and the FTS5 keyword table is not cleaned during consolidation, creating a latent inconsistency between the two episodic backends ($i \in S$ but $i \notin C$ after deletion).
7) **Fixed decay rate.** $\lambda = 0.02$ in Eq. 1 is a single hardcoded value, not tuned per user or per deployment.
8) **No cross-memory inference.** The three subsystems operate independently; the architecture cannot yet infer a semantic fact (e.g., `HAS_SKILL: Python`) purely from a recurring episodic pattern.
9) **Scalability.** SQLite is adequate for single-node deployments but is a known bottleneck under concurrent multi-tenant load; we have not load-tested the system.
10) **Known extraction edge case.** The role-update HTTP 500 errors described in Section XI indicate the semantic extraction pipeline is not yet hardened against multi-fact single-turn updates.

We consider items 1 and 2 the highest-priority next steps: without benchmark-grade, ablated evidence, comparative claims relative to Mem0, Zep, or MemGPT should be read as architectural positioning (Table I) rather than performance claims.

## XIV. Future Work

Immediate priorities are: (1) evaluation on LongMemEval and LoCoMo against flat-RAG, Mem0, and Zep baselines under matched compute budgets; (2) an ablation study isolating dual-storage retrieval, cardinality typing, and the Ground Truth Hierarchy prompt; (3) a relevance-based pruning or periodic summarization mechanism for 1:$N$ semantic relations; (4) multi-backend LLM support with graceful fallback; and (5) exploration of graph neural network-based cross-memory inference to derive implicit semantic facts from recurring episodic patterns, as originally outlined as future work.

## XV. Conclusion

We presented TriMem, an open, cognitively-inspired tripartite memory architecture for conversational AI agents that separates episodic experience, semantic fact, and procedural rule into distinct stores with a cardinality-typed ontology and an explicit, deterministic precedence rule for resolving cross-memory conflicts. We described the architecture, its retrieval and consolidation algorithms, and reported internal validation results, while being explicit about which of our observations are hard structural guarantees (semantic 1:1 overwrite) versus empirical, model-dependent behavior (cross-subsystem precedence, consolidation quality). We believe the architecture's main value is as a reproducible systems pattern, and we have outlined the benchmark-based evaluation and ablation work needed before stronger comparative claims can be supported.

## References


[1] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proc. ACM UIST*, 2023.

[2] C. Packer, S. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," *arXiv:2310.08560*, 2023.

[3] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.

[4] L. Wang *et al.*, "Augmenting Language Models with Long-Term Memory," in *Proc. ACL*, 2023.
[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," in *Proc. ECAI*, 2025, arXiv:2504.19413.
[6] P. Rasmussen *et al.*, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," *arXiv preprint*, 2025.
[7] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-Mem: Agentic Memory for LLM Agents," *arXiv:2502.12110*, 2025.
[8] W. Zhong *et al.*, "MemoryBank: Enhancing Large Language Models with Long-Term Memory," in *Proc. AAAI*, 2024.
[9] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating Very Long-Term Conversational Memory of LLM Agents," *arXiv:2402.17753*, 2024.
[10] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," *arXiv:2410.10813*, 2025.
[11] E. Tulving, "How Many Memory Systems Are There?," *American Psychologist*, vol. 40, no. 4, pp. 385–398, 1985.
[12] Anonymous, "ZenBrain: A Neuroscience-Inspired 7-Layer Memory Architecture for Autonomous AI Systems," *arXiv preprint*, 2026.